

8-4 · Streamlit 기반의 챗봇 인터페이스 개발 · 실습 1.5H

챗봇 화면 구현

에코 챗봇으로 뼈대를 익히고 재실행을 눈으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

- ① rerun_demo.py 를 눌러 보며 재실행을 직접 확인한다
- ② chat_echo.py 로 말풍선이 쌓이는 에코 챗봇을 만든다
- ③ 다시 그리는 루프를 지워 대화가 사라지는 것을 재현한다
- ④ 사이드바·스트리밍·@st.cache_resource 를 붙여 본다

화면을 직접 눌러 보는 것이 오늘 이해의 핵심입니다 — 코드만 봐서는 남지 않습니다

일곱 단계 — ~/rag-chatbot 한 폴더에서

새 폴더에 가상환경을 만들고 시작합니다 — 이 폴더가 오늘 챗봇의 집입니다

1. 준비

venv ·
pip install

2. 재실행 확인

rerun_demo.py

3. 에코 챗봇

chat_echo.py

4. 일부러 실패

for 루프
주석 처리

5~7. 확장

사이드바 ·
스트리밍 · 캐시

1단계

프로젝트 준비 — 설치와 키 등록

폴더	<code>mkdir -p ~/rag-chatbot && cd ~/rag-chatbot</code>
가상환경	<code>python -m venv .venv · source .venv/bin/activate</code>
설치	<code>pip install streamlit qdrant-client openai</code>
무시 목록	<code>.gitignore</code> 에 <code>.venv/</code> 와 <code>.env</code>
키	<code>export OPENAI_API_KEY=sk-본인키로교체</code>
확인	<code>streamlit --version</code> → 버전 문자열이 나오면 완료

`export` 는 그 터미널에만 적용된다 — 새 터미널을 열면 다시 해야 한다 · 키 값을 코드에 넣지 않는다

재실행을 눈으로 — 버튼 누를 때마다 시각이 바뀐다

1 streamlit run rerun_demo.py
시각 · 일반 변수 카운터 · session_state 카운터 세 줄이 뜬다

2 버튼을 누를 때마다 시각이 바뀐다
스크립트가 처음부터 다시 돌았다는 증거

3 일반 변수는 언제나 1, session_state 만 쌓인다
이 화면을 직접 눌러 보는 것이 오늘 이해의 핵심

바뀌는 것

시각 ·
n 값

counter = 0 은 재실행마다 0 으로 돌아가고, if "n" not in st.session_state: 로 만든 n 만 1, 2, 3 으로 는다

chat_echo.py — 뼈대 네 단계가 코드로

뼈대	코드
이력 저장소	if "messages" not in st.session_state:
이력 다시 그리기	for m in st.session_state.messages:
입력 받기	prompt = st.chat_input(...)
사용자 말풍선	with st.chat_message("user"):
답 만들기	answer = "따라 말하기: " + prompt
답 말풍선	with st.chat_message("assistant"):

streamlit run chat_echo.py → localhost:8501 에 채팅 화면 · 여러 번 입력해도 이전 말풍선이 쌓이면 성공

for 루프를 지우면 — 이력은 있는데 화면엔 없다

루프 있음 — 정상

재실행마다 이력 전체를 다시 그린다



VS

루프 주석 처리 — 실패

이력은 session_state 에 그대로 — 확인 뒤 주석 해제



사이드바 초기화 버튼과 흘러나오는 글자

1 with st.sidebar: 설정 · 대화 N턴 · 지우기
지우기 버튼 → `st.session_state.messages = []` 뒤 `st.rerun()`

2 st.rerun() — 코드에서 재실행을 부른다
없으면 다음 상호작용까지 화면이 안 바뀐다

3 stream_text(text) — 한 글자씩 yield
`time.sleep(0.01)` 로 글자가 하나씩 나타난다

4 answer = st.write_stream(...)
흘러나온 전체 문자열을 돌려주니 그대로 이력에 넣는다

같은 방식

**LLM 스트리밍
응답에도**

@st.cache_resource — 3초가 한 번만

데코레이터 있음

처음 3초, 이후 재실행에서는 즉시 지나간다

- heavy_setup() 안에 time.sleep(3)
- 준비 시작·끝 시각 차이 3초 — 처음만
- 다시 누르면 즉시 지나간다

VS

데코레이터 지움

재실행마다 3초씩 걸린다

- 같은 함수, 데코레이터 한 줄만 삭제
- 버튼을 누를 때마다 3초
- 되돌려 재 보면 캐시가 하는 일이 분명해진다

이 1.5시간이 끝날 때 확인할 것

- 1 **streamlit --version** 이 버전 문자열을 출력한다
- 2 **rerun_demo.py** 에서 버튼마다 시각이 바뀌고 **session_state** 만 쌓인다
- 3 **chat_echo.py** 에서 여러 번 입력해도 이전 말풍선이 남는다
- 4 **for** 루프를 지우면 이전 대화가 사라지는 것을 봤다
- 5 **cache_resource** 를 지우면 매번 3초, 붙이면 처음만 3초

8-4 · 실습 1.5H 마무리

다음은 Qdrant 검색·LLM 연동 실습 (1H)

에코 자리에 질문 임베딩 → Qdrant 검색 → LLM 답변 3단계를 넣어 실제로 답하는 챗봇을 만듭니다

✓ curl 로 lecture_docs 의 points_count 확인

✓ search_test.py 로 검색만 먼저 시험

✓ app.py 에 RAG 3단계 연결

✓ 근거 표시·개수 조절·약한 근거 경고

가져갈 것

chat_echo.py 가 돌아가는 터미널과 키